

MoviScript

Lenguaje específico de dominio para programar móviles

Pablo Dobry

October 14, 2024

1 Descripción general

El lenguaje MoviScript es un lenguaje específico de dominio (DSL) orientado a la programación de móviles como el recorrido de pequeños autos eléctricos programables y robots. Este lenguaje tiene la finalidad de facilitar la programación de la movilidad, ofreciendo una sintaxis intuitiva y funcionalidades específicas para abordar los desafíos asociados con estos dispositivos. Este DSL está diseñado para simplificar tareas comunes en la programación del desplazamiento, promoviendo un desarrollo eficiente.

Este lenguaje cuenta con comandos clásicos de un lenguaje imperativo, como la posibilidad de operar sobre expresiones matemáticas y booleanas, condicionales y bucles. Cuenta con instrucciones específicas de la programación de móviles como la posibilidad de generar una lista de puntos en función de una expresión matemática y una lista de puntos donde evaluarla.

Cuenta con instrucciones para el seguimiento de un camino (una lista de puntos), como así también la posibilidad de que, en caso de verse impedido el avance del móvil elabore un plan de contingencia para poder llegar al siguiente punto del camino.

La implementación de un lenguaje, el DSL se construye directamente embebido dentro de Haskell, utilizando sus funcionalidades como base. Luego, se emplea la biblioteca Parsec de Haskell para llevar a cabo el análisis sintáctico del código fuente del DSL.

Una vez completado el paso de parseo, se procede a la construcción de un Árbol de Sintaxis Abstracta (AST). Este árbol representa la estructura jerárquica del código fuente y sirve como una representación interna que facilita la manipulación y evaluación del código.

Finalmente, para la evaluación semántica del DSL, se recurre a un evaluador monádico. Las mónadas en Haskell ofrecen un mecanismo para modelar cálculos con efectos secundarios, lo que resulta esencial para la interpretación y ejecución de expresiones en nuestro lenguaje.

2 Manual de uso

El lenguaje Movi esta embebido en Haskell por lo que para compilar el lenguaje MoviScript se deberá tener instalado un compilador de **Haskell**. También es necesario tener instalado el programa **ucblogo** para poder ejecutar el script en logo con el que se evalúa el programa. Además, es necesario instalar **Python** y la biblioteca **Matplotlib** para ver los puntos de la traza. Para hacer estas instalaciones se puede ejecutar en una distribución Debian o Ubuntu el siguiente comando:

```
apt-get update
apt-get install ghci
apt-get install ucblogo
```

Además, es necesario instalar Python y la biblioteca Matplotlib para ver los puntos de la traza.

```
apt-get install python3 python3-pip
pip3 install matplotlib
```

Para ejecutar un programa MoviScript se debe ejecutar el script `movi.sh` en la misma carpeta en la que se encuentran los archivos `movi.sh`, `Main.hs`, `Parser.hs` y `EvalMovi.hs`:

```
./movi.sh <ruta-del-programa>/miPrograma.movi
```

De esta forma se ejecuta el programa *miPrograma.movi*. La ejecución en esta primera versión de MoviScript producirá una salida en pantalla con las primitivas ejecutadas por el móvil y la traza con los puntos por los que pasó el móvil y un gráfico de la traza realizado en logo.

3 Expresiones flotantes

El lenguaje incorpora funciones trigonométricas (seno, coseno, tangente), logaritmo, redondeo hacia abajo (floor) y redondeo hacia arriba (ceil), ofreciendo así una amplia gama de herramientas matemáticas. La inclusión de funciones especializadas como la exponencial y el redondeo a un entero mediante la función 'round' brinda flexibilidad a los desarrolladores para expresar de manera eficiente y concisa operaciones matemáticas complejas. Esta sintaxis proporciona una base robusta para la manipulación de cálculos numéricos en el contexto de programación de móviles.

La sintaxis del lenguaje es simple y se describe a continuación.

- *expf* : Negación de una expresión flotante (*expf*).
- (*expf*) : Agrupación de una expresión flotante.
- expf* + *expf* : Suma de dos expresiones flotantes.
- expf* * *expf* : Multiplicación de dos expresiones flotantes.
- expf* - *expf* : Resta de dos expresiones flotantes.
- expf* / *expf* : División de dos expresiones flotantes.
- sen** (*expf*) : Seno de una expresión flotante.
- cos** (*expf*) : Coseno de una expresión flotante.
- tan** (*expf*) : Tangente de una expresión flotante.
- log** (*expf*) : Logaritmo de base 10 una expresión flotante.
- floor** (*expf*) : Redondeo hacia abajo de una expresión flotante.

ceil (*expf*) : Redondeo hacia arriba de una expresión flotante.

exp (*expf*, *expf*) : Función exponencial con dos argumentos flotantes.

round (*expf*, *num*) Redondeo de una expresión flotante con una cantidad determinada (*num*) de decimales.

4 Expresiones booleanas

Las expresiones booleanas se pueden relacionar con los siguientes operadores.

true y **false**: Representan valores booleanos directos.

expf < *expf* y *expf* > *expf*: Permite comparaciones relacionales entre expresiones flotantes.

boolexp & *boolexp*: Operador lógico AND para conjunción de expresiones booleanas.

boolexp | *boolexp*: Operador lógico OR para disyunción de expresiones booleanas.

~ *boolexp*: Operador unario NOT para negación de una expresión booleana.

(*boolexp*): Permite agrupar expresiones booleanas para establecer el orden de evaluación.

5 Comandos básicos y de control de flujo

En este lenguaje, se emplean comandos fundamentales para la manipulación eficiente de variables y el control del flujo de ejecución típicos en cualquier lenguaje imperativo.

varf := *expf*

La asignación de expresión flotante (*expf*) a una variable de tipo flotante (*varf*) se hará con el operador :=. En MoviScript sólo se permitirá el almacenamiento de valores flotantes en las variables. En versiones futuras se podrán incorporar variables de punto, de listas de expresiones flotantes y de listas de punto.

comm ; *comm* : La secuenciación de comandos se realiza con el carácter ';'.

if (*boolexp*) **then** *comm* **else** *comm* **end**

Estructura condicional clásica if-then-else que ejecuta un bloque de comandos u otro según la evaluación de una expresión booleana.

while (*boolexp*) **do** *comm* **end**

Establece un bucle que repite la ejecución de un bloque de comandos mientras una condición booleana se mantenga verdadera.

6 Comandos sobre puntos y listas de puntos

En MoviScript, las instrucciones relacionadas con puntos proporcionan herramientas fundamentales para la manipulación espacial y el análisis geométrico. Estas instrucciones permiten la operaciones de puntos en un plano bidimensional como la medición de la distancia entre el origen y un punto específico, así como la determinación del ángulo que dicho punto forma con el eje x. La capacidad de calcular la distancia y el ángulo facilita el desarrollo de algoritmos espaciales avanzados, permitiendo a los programadores abordar tareas relacionadas con la ubicación y la orientación en el contexto específico de dispositivos móviles.

La representación de un punto en nuestro lenguaje sera un par ordenado de expresiones flotantes.

$$point = (expf, expf)$$

Sobre un punto se pueden realizar los siguientes comandos.

dist(*point*). Establece la distancia entre un punto (*point*) y el origen de coordenadas

$$\mathbf{rangle}(point)$$

El comando *rangle* establece el angulo (en radianes, de 0 a 2π) determinado entre la recta que pasa por el punto (*point*) y el eje x.

$$\mathbf{gangle}(point)$$

El comando *gangle* establece el angulo (en grados, de 0 a 360°) determinado entre la recta que pasa por el punto (*point*) y el eje x.

$$\mathbf{path}(expf, var, [expf])$$

Con el comando *path* podremos generar listas de puntos (x_i, y_i) en función de una expresión matemática (*f*), una variable (*x*) y una lista de expresiones flotantes donde evaluaremos nuestra expresión matemática (x_i). El comando toma tres argumentos:

- *expf* : Es una expresión matemática (*f*) que generara los valores y_i .
- *var* : Es la variable (*x*) en que se evaluará en la expresión matemática *expf*.
- [*expf*]: Es la lista de valores x_i en la que se evaluará la función *f*.

7 Comandos de movimientos

En esta sección describiremos las funciones que brindan la capacidad del dispositivo para moverse de manera inteligente y eficiente, ofreciendo a los programadores las herramientas necesarias para la creación de algoritmos de movilidad. Por un lado tenemos herramientas de navegación básica:

$$\mathbf{fd}(dist, time)$$

Con el comando *Fd* se indica al móvil el avance lineal de una distancia(*dist*, de tipo expresión flotante) en durante en un tiempo determinado(*time*, de tipo expresión flotante).

$$\mathbf{bk}(dist, time)$$

Con el comando *Bk* se indica al móvil el retroceso lineal de una distancia(*dist*, de tipo expresión flotante) en durante en un tiempo determinado(*time*, de tipo expresión flotante).

$$\mathbf{turn}(ang, time)$$

Con el comando *Turn* se indica al móvil el giro de un ángulo en grados(*ang*, de tipo expresión flotante) en durante en un tiempo determinado(*time*, de tipo expresión flotante).

El lenguaje, también cuenta con instrucciones complejas que permiten elaboración de algoritmos avanzados:

lookat(*point*, *vel*)

Con el comando *LookAt* se indica al móvil el giro en dirección al punto (*point*). La coordenada del punto será en caracter relativo al propio móvil tomando como origen de coordenadas el móvil y el sentido positivo del eje x será la dirección en la que esta apuntando el móvil al momento de ejecutar la instrucción. La velocidad de giro será la indicada como *vel*.

goline(*point*, *vel* – *lineal*, *vel* – *angular*)

Con el comando *GoLine* se indica al móvil el giro en dirección al punto (*point*) y luego su desplazamiento al punto mencionado. La coordenada del punto será en caracter relativo al propio móvil tomando como origen de coordenadas el móvil y el sentido positivo del eje x será la dirección en la que esta apuntando el móvil al momento de ejecutar la instrucción. La velocidad de giro será la indicada como *vel* – *angular* y la velocidad de desplazamiento será la indicada como *vel* – *lienal*.

follow(*listpoint*, *vel* – *lineal*, *vel* – *angular*)

Con el comando *follow* se indica que el móvil haga un recorrido determinado por la lista de puntos (*listpoint* = [(*x*₁, *y*₁), (*x*₂, *y*₂)...(*x*_{*n*}, *y*_{*n*})]).

La velocidad de giro será la indicada como *vel* – *angular* y la velocidad de desplazamiento será la indicada como *vel* – *lienal*. Los puntos del camino están en relación a al punto de referencia que es el móvil al momento de iniciarse la ejecución del programa.

followsmart(*listpoint* – *allow*, *listpoint* – *contingency* – *allow*, *vel* – *lineal*, *vel* – *angular*)

De igual manera que el comando *follow*, el comando *followsmart* indica que el móvil haga un recorrido determinado por la lista de puntos seguido de las posiciones en las que estará habilitado el móvil para circular. La lista de puntos-contingency será la lista de puntos (*listpoint* – *allow* = *def_obstacles*[(*x*₁, *y*₁), (*x*₂, *y*₂)...(*x*_{*n*}, *y*_{*n*})], [*x*₁...*x*_{*i*}...*x*_{*n*}]) donde *x*_{*i*} indica la lista de puntos en los que el móvil esta obstaculizado por el medio para circular.

En este caso, el comando *followsmart* incorpora información del medio. Lo hace mediante la lista de puntos que en este caso será una lista de índices en los que los puntos obstaculizados por el medio.

Si el móvil intentar circular por una parte del camino que está impedido por algún obstaculo el lenguaje MoviScript tendrá un camino de contingencia. Al igual que en el caso de *follow*, los puntos del camino están en relación a al punto de referencia que es el móvil al momento de iniciarse la ejecución del programa. Sin embargo, la lista de contingencia es relativa al último punto antes de encontrarse un obstaculo y se utilizará como contingencia de forma recursiva, de forma de que si hay obstaculos abordando el camino de contingencia el móvil también utilizará el camino de contingencia.

8 Semántica de los comandos

Fd

$$\frac{p \downarrow (p_x, p_y)}{(\text{Fd}(d, v), \sigma(\theta_{\text{old}}, p)) \rightarrow (\text{Skip}, \sigma(\theta_{\text{old}}, (p_x + d \cdot \cos\left(\frac{\theta \cdot \pi}{180.0}\right), p_y + d \cdot \sin\left(\frac{\theta \cdot \pi}{180.0}\right))))}$$

Turn

$$\overline{\text{Turn}(\theta, v), \sigma(\theta_{\text{old}})} \rightarrow \overline{\text{Skip}, \sigma(\theta_{\text{old}} + \theta)}$$

Lookat

$$\frac{p \downarrow (p_x, p_y)}{\text{Lookat}(p, v) \rightarrow \text{Turn}(\frac{180}{\pi} * \text{atan}^2(p_x, p_y), v)}$$

Goline

$$\frac{p \downarrow \text{dist}}{\text{Goline}(p, v_1, v_2) \rightarrow \text{Seq}(\text{Lookat}(p, v_1), \text{Fd}(\frac{\text{dist}}{v_2}, v_2))}$$

TurnAbs

$$\overline{\text{TurnAbs}(\text{ang}, \text{vel}) \rightarrow \text{Turn}\left(\left(\frac{\text{ang} - \text{angAct}}{v}\right), \text{vel}\right)}$$

GolineAbs

$$\overline{\text{GolineAbs}((p_x, p_y), v_1, v_2) \rightarrow \text{Seq}(\text{TurnAbs}(\frac{180}{\pi} \cdot \text{atan}^2(p_y, p_x)) \ v_1, \text{Fd}(\frac{\text{dist}}{v_2}, v_2))}$$

Follow

$$\overline{\text{Follow}([], v_1, v_2) \rightarrow \text{Skip}} \quad (1)$$

$$\overline{\text{Follow}([p], v_1, v_2) \rightarrow \text{GolineAbs}(p, v_1, v_2)} \quad (2)$$

$$\overline{\text{Follow}(p : ps, v_1, v_2) \rightarrow \text{Seq}(\text{GolineAbs}(p, v_1, v_2), \text{Follow}(ps, v_1, v_2))} \quad (3)$$

$$\frac{\text{Path}(\text{exp}, v, \text{list}) \downarrow xs}{\text{Follow}(\text{Path}(\text{exp}, v, \text{list}), v_1, v_2) \rightarrow \text{Follow}(xs, v_1, v_2)} \quad (4)$$

Followsmart

$$\overline{\text{FollowSmart}([], \text{contingency}, v_1, v_2) \rightarrow \text{Skip}} \quad (1)$$

$$\overline{\text{FollowSmart}([(p, \text{True}) : xs], \text{contingency}, v_1, v_2) \rightarrow \text{Seq}(\text{GolineAbs}(p, v_1, v_2), \text{FollowSmart}(xs, \text{contingency}, v_1, v_2))} \quad (2)$$

$$\overline{\text{FollowSmart}([(p, \text{False}) : xs], \text{contingency}, v_1, v_2) \rightarrow \text{Skip}} \quad (3)$$

$$\frac{\text{isAllFalse}(\text{contingency})}{\text{FollowSmart}([(p, \text{False}) : xs], \text{contingency}, v_1, v_2) \rightarrow \text{FollowSmart}(xs, [], v_1, v_2)} \quad (4)$$

$$\overline{\text{FollowSmart}([(p, \text{False}) : xs], \text{contingency}, v_1, v_2) \rightarrow \text{FollowSmart}(\text{contingency} ++ xs, \text{contingency}, v_1, v_2)} \quad (5)$$

$$\overline{\text{FollowSmart}(\text{def_obstacles}(ps, \text{list}), \text{contingency}, v_1, v_2) \rightarrow \text{FollowSmart}(xs, \text{contingency}, v_1, v_2)} \quad (6)$$

$$\frac{def_obstacles(ps, list) \downarrow xs}{FollowSmart(listpoints, def_obstacles(ps, list), v_1, v_2) \rightarrow FollowSmart(listpoints, xs, v_1, v_2)} \quad (7)$$

isAllFalse

$$\frac{}{isAllFalse ([]) \rightarrow True}$$

$$\frac{}{isAllFalse ((x, b) : xs) \rightarrow \neg b \wedge isAllFalse(xs)}$$

9 Posibles próximas versiones

En futuras versiones, MoviScript podrá tomar información del medio con un sensor frontal del móvil. También habrá posibilidad de almacenamiento de más tipos de variables como las listas, los puntos y la lista de obstaculos. También se podrá compilar MoviScript a código *NQC*¹ y de esta forma podrá ser ejecutado en un móvil construido en *LEGOMINDSTORMS*.

¹Not Quite C es un lenguaje simple que sintaxis similar a C utilizado para programar la línea robótica de LEGO denominada Maindstorms. Para más información visitar la documentación: <https://bricxcc.sourceforge.net/nqc/>